VidyaTrack Frontend Junior Developer Onboarding Guide

1. Why This Document Exists

This guide is for junior frontend developers joining the VidyaTrack codebase.

You should be able to use this document to understand:

- where to start reading the app
- how the frontend is organized
- how data is fetched
- how to add or change a feature safely
- how to avoid common mistakes in this repo

2. First Mental Model

Think of the app in this order:

1. router chooses the role area
2. layout renders the shell
3. page decides what screen to show
4. hook controls page behavior
5. API fetches data
6. helper/utils transform data
7. components render UI

If you keep that model in mind, most of the repo will feel consistent.

3. What Stack We Use

- React
- TypeScript
- React Router
- React Query
- Zustand
- React Hook Form
- Tailwind

- Axios

What each one is used for:

- React: UI
- TypeScript: typing and safer refactors
- Router: page navigation
- React Query: server data
- Zustand: auth and selected school context
- React Hook Form: form state
- Tailwind: styling
- Axios: HTTP requests

4. Important Folders

`src/app`

Use this to understand how the whole application is assembled.

Key files:

- `src/app/router.tsx`
- `src/app/routes/*`

`src/features`

This is where most product work happens.

Each feature usually has:

- `components`
- `hooks`
- `helpers`
- `utils`
- `types`
- `constants`

`src/hooks`

Shared hooks used by multiple features.

`src/api`
All backend requests.

`src/store`
Global app state.

`src/components`
Reusable UI building blocks.

5. How To Read a Feature

When reading a feature, use this order:

1. route shell
2. feature hook
3. components
4. helpers
5. utils
6. types/constants

Example:

- read `TeacherCommunicationsPage.tsx`
- then `useTeacherCommunications.ts`
- then the tab panel components
- then helper and util files

6. What Belongs Where

Put code in `components/` when:

- it renders UI
- it should be controlled via props
- it has little or no side effects

Put code in `hooks/` when:

- it uses React state

- it performs effects
- it runs queries or mutations
- it manages orchestration

Put code in `helpers/` when:

- it is pure business logic
- it validates rules
- it derives domain state

Put code in `utils/` when:

- it formats text, dates, numbers
- it maps data shapes
- it is pure and generic

Put code in `constants/` when:

- it is a fixed list
- it is a stable label/config value
- it is a shared query key helper

7. Understanding Data Fetching

We use React Query for backend data.

That means:

- query hooks fetch data
- mutation hooks write data
- cache invalidation refreshes stale data

Important file:

- `src/constants/queryKeys.ts`

Always use shared query keys when possible.

Do not:

- invent new inline query keys unless there is a strong reason
- invalidate broad keys if a smaller scoped key exists

8. Understanding School Context

VidyaTrack is multi-tenant by school.

That means many APIs need a school ID.

Shared helpers:

- `src/helpers/requireSchoolId.ts`
- `src/api/helpers/schoolParams.ts`

Rules:

- use `requireSchoolId` in mutations or flows that must have a selected school
- use `schoolParams` when passing `school_id` to APIs

9. Understanding Auth and Role Access

Global auth state lives in Zustand.

What it usually contains:

- role
- schoolId
- schools
- role/school mappings

Routes are protected with `RoleGuard`.

So if you add a new role-specific page, it should usually go in the correct role route module.

10. Common Page Pattern

A normal page now looks like this:

- route shell imports the hook and some components
- hook returns data, loading state, error state, handlers, derived values
- components render sections using props only

This is intentional.

Do not move lots of logic back into page JSX.

11. How To Add a New Query Hook

Checklist:

1. define the API request in `src/api`
2. add a shared query key in `src/constants/queryKeys.ts` if it is reusable
3. create the hook in `src/hooks` or feature-local `hooks`
4. keep `enabled` logic explicit
5. return stable shape: `data`, `isLoading`, `error`, `refetch` if applicable

12. How To Add a New Mutation Hook

Checklist:

1. define API call
2. assert school context if needed
3. perform mutation in hook
4. invalidate the smallest correct query scope
5. return mutation state the feature needs

13. Forms in This Repo

If a page uses React Hook Form:

- field rendering stays in components
- submit logic stays in hooks
- payload shaping happens in hooks

Do not let the JSX page build complex API payloads inline.

14. How To Debug a Feature

Use this order:

1. confirm route file is correct
2. inspect the feature hook return values
3. inspect the API hook/query state
4. inspect query key identity
5. inspect selected school context

6. inspect helper derivations

Most bugs in this codebase come from:

- wrong role/school context
- stale invalidation scope
- incorrect selected entity state
- local derivation mismatch

15. How To Explain One Feature to Yourself

Ask:

- what is the route?
- what data does it fetch?
- what local state does it own?
- what mutations can it run?
- what derived values does it compute?
- which parts are UI-only?

If you can answer those six questions, you understand the feature.

16. Good Engineering Habits in This Repo

- keep route files short
- keep components pure
- keep effects in hooks
- reuse shared query keys
- preserve user behavior during refactors
- prefer extraction over rewrite

17. Bad Patterns To Avoid

- business logic in JSX
- ad hoc query keys
- unscoped invalidation
- repeating `school_id` param code in many places
- putting feature-only code in top-level shared folders too early

18. Suggested Learning Path

Read these in order:

1. `src/main.tsx`
2. `src/app/router.tsx`
3. `src/app/routes/*`
4. `src/layouts/*`
5. `src/store/auth.store.ts`
6. `src/constants/queryKeys.ts`
7. one feature from each role

Recommended features to study:

- teacher dashboard
- teacher communications
- management students
- principal communications
- platform create school

19. First Real Tasks for a Junior Developer

Good starter tasks:

- small UI copy change in an extracted component
- add a filter chip to a list page
- add a derived badge using helpers
- add a small query-key-backed hook
- add empty/loading state refinement without changing behavior

20. Before Opening a PR

Do this every time:

1. run lint
2. run build
3. verify query keys and invalidations
4. confirm no user-facing behavior changed unintentionally
5. check if any new code should live in helpers/utils/components/hooks instead of one file

21. Practical Summary

If you forget everything else, remember this:

- pages compose
- hooks orchestrate
- components render
- helpers decide rules
- utils format data
- APIs fetch
- query keys identify cache
- school context matters almost everywhere

That is enough to work productively in this frontend.